Signals and Systems Fundamentals

E C E 2 5 1 s



Course Project Spring 26

Digital Audio Signal Processing Using MATLAB

Name	ID
Youssef Hatem Farid Ahmed	2400453
Habeba Hosam El Din Sayed	2400307
Wesam Essam Mohamed Sayed	2400913
Zeina Mostafa Ali El-Bayoumi	2400421
Marwan Hamed Ahmed Hamed	2400607
Ghada Yehia Ibrahim Hamed	2400831

Introduction

This project aims to analyze and process an audio signal using MATLAB. The program reads an audio file, converts it to a mono signal if necessary, and represents the signal in both time and frequency domains using the Fast Fourier Transform (FFT). The signal energy is calculated in both domains to verify Parseval's theorem and ensure consistency between the results. The dominant frequency component of the signal is then identified, and an ideal notch filter is applied around this peak frequency to remove selected frequency components. Finally, the filtered signal is transformed back into the time domain using the inverse FFT (IFFT), plotted, and saved as a new audio file.

Filtration Process

In this project, a bandwidth filter was applied around the dominant frequency component to reduce or remove selected frequency ranges from the audio signal. Different bandwidth values of **50 Hz, 100 Hz, and 200 Hz** were tested to observe their effect on the signal.

- **50 Hz bandwidth:** removes a very narrow frequency range, causing minimal change to the original audio.
- **100 Hz bandwidth:** removes a wider range around the target frequency, resulting in moderate filtering.
- **200 Hz bandwidth:** removes a much larger frequency range, producing stronger filtering and greater changes in the audio quality.

Signal Processing

Section 1: the program prompts the user to select an audio file (WAV) using a graphical interface. The signal is then read into the workspace, converted to mono if it was stereo by averaging the channels, and the time vector is established based on the sampling frequency (F_s)

```
clc;
clear;
close all;


% 1) Read Audio File



[fileName, pathName] = uigetfile({'*.wav;*.mp3'}, 'Select Audio Fil

audioPath = fullfile(pathName, fileName);

[x, Fs] = audioread(audioPath);
```

Section 2: Plot Time Domain Signal

This part visualizes the raw audio signal in the time domain, showing the amplitude variation over time. This step is crucial for an initial inspection of the signal's waveform and duration.

```
% 2) Plot Time Domain Signal


figure;
plot(t, x);
xlabel('Time (seconds)');
ylabel('Amplitude');
title('Audio Signal in Time Domain');
grid on;
```

Section 3: Energy in Time Domain

"The total energy of the signal is calculated in the time domain by summing the squares of the signal's absolute amplitudes. This provides a baseline value for energy conservation verification."

```
% 3) Energy in Time Domain


Energy_time = sum(abs(x).^2);

fprintf('Energy from Time Domain = %.4f\n', Energy_time);

```

Section 4: FFT and Frequency Domain

The signal is transformed from the time domain to the frequency domain using the Fast Fourier Transform (FFT). The fftshift function is applied to center the zero-frequency component, and a linear frequency axis (f) is created ranging from $-\frac{Fs}{2}\ to \frac{Fs}{2}$.

```
% 4) FFT and Frequency Domain

x = x(:);          % force column vector

N = length(x);

X = fft(x);

X_shift = fftshift(X);

f = linspace(-Fs/2, Fs/2, N)';

```

Section 5: Plot Frequency Spectrum

The magnitude spectrum is plotted to visualize the frequency content of the audio. This plot helps identify the dominant frequencies and the overall distribution of the signal's power across the spectrum.

```
% 5) Plot Frequency Spectrum


figure;
plot(f, abs(X_shift));
xlabel('Frequency (Hz)');
ylabel('Magnitude');
title('Frequency Domain Representation');
grid on;

```

Section 6: Energy in Frequency Domain (Parseval's Theorem)

To verify the numerical accuracy of the transform, the signal energy is recalculated in the frequency domain. According to Parseval's Theorem, this value (Energy _ freq) should match the time-domain energy, confirming that the transformation process preserved the signal's total energy.

Parseval's Theorem

The energy calculated in the time domain is equal to the energy calculated in the frequency domain according to Parseval's theorem.

$$\sum_{n=0}^{N-1} | x[n] |^2 = \frac{1}{N} \sum_{k=0}^{N-1} | X[k] |^2$$

where:

- $x[n]$ *is the signal in time domain*
- $X[k]$ *is the FFT of the signal*
- N *is the total number of samples*

The values obtained from both domains were nearly identical, which verifies the correctness of the FFT implementation.

```
% 6) Energy in Frequency Domain
% Parseval Theorem


Energy_freq = (1/N) * sum(abs(X).^2);

fprintf('Energy from Frequency Domain = %.4f\n', Energy_freq);

```

Section 7: Find Peak Frequency

The algorithm automatically identifies the peak frequency (f_{peak}) by finding the index of the maximum magnitude in the shifted spectrum. This frequency represents the most dominant tone in the recorded audio, which will be the target for the notch filter.

```
81
82      % 7) Find Peak Frequency
83      ≅
84
85      [magnitude_max, index_max] = max(abs(X_shift));
86
87      f_peak = f(index_max);
88
89      fprintf('Peak Frequency = %.2f Hz\n', f_peak);
90
```

Section 8: Ideal Notch Filter Design & Application

A loop is implemented to test three different bandwidths for the Ideal Notch Filter. For each bandwidth, a frequency response H is created (equal to 1 everywhere and 0 at the notch region around ± f_{peak}). The filter is applied by performing point-by-point multiplication in the frequency domain.

```
92      % 8) Ideal Notch Filters
93      % Try 3 different bandwidths
94
95
96      bandwidths = [50 100 200];
97
98      for k = 1:length(bandwidths)
99
100         BW = bandwidths(k);
101
102         % Create filter
103         H = ones(size(f));
104         % Notch region around +f_peak
105         H(abs(f - f_peak) < BW/2) = 0;
106
107         % Notch region around -f_peak
108         H(abs(f + f_peak) < BW/2) = 0;
109
110         % Apply filter
111         X_filtered = X_shift .* H;
```

Section 9: Convert Back to Time Domain

The filtered spectrum is shifted back using ifft shift and then transformed back to the time domain using the Inverse Fast Fourier Transform (IFFT). Only the real part of the result is retained to eliminate any negligible imaginary components resulting from numerical precision limits.

```
Plot filtered spectrum

112      figure;
113      plot(f, abs(X_filtered));
114      xlabel('Frequency (Hz)');
115      ylabel('Magnitude');
116      title(['Filtered Spectrum - BW = ', num2str(BW), ' Hz']);
117      grid on;
118
119
120      % 9) Convert Back to Time Domain
121
122
123      X_ifftshift = ifftshift(X_filtered);
124
125      x_filtered = real(ifft(X_ifftshift));
```

Section 10: Plot & Save Filtered Audio

The final filtered signal is plotted in the time domain for visual comparison and then saved as a new WAV file. The filename dynamically includes the bandwidth value to distinguish between the different filtering results for further analysis.

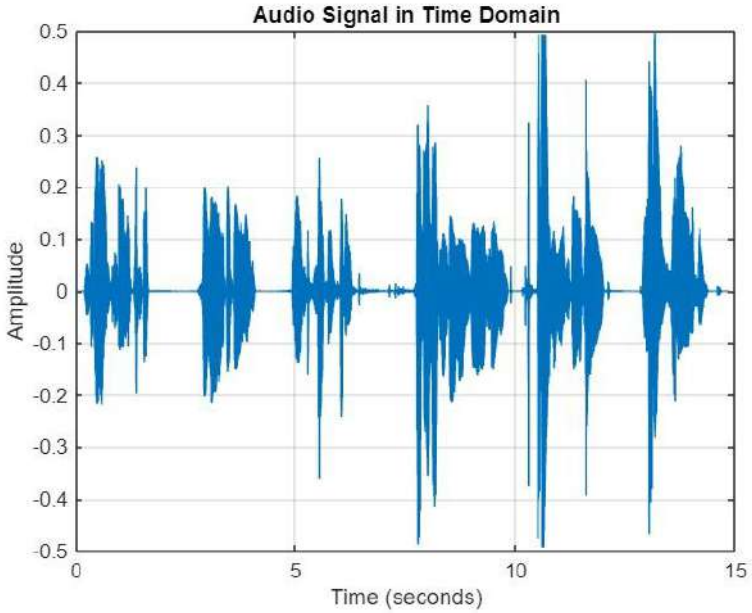
```
Plot filtered signal in time domain

126      figure;
127      plot(t, x_filtered);
128      xlabel('Time (seconds)');
129      ylabel('Amplitude');
130      title(['Filtered Signal in Time Domain - BW = ', num2str(BW), ' Hz']);
131      grid on;
132
133
134      % 10) Save Filtered Audio
135
136
137      outputFile = ['filtered_BW_', num2str(BW), '.wav'];
138
139      audiowrite(outputFile, x_filtered, Fs);
140
141      fprintf('Saved: %s\n', outputFile);
142
143  end
144
145  disp('signal filtered');
```

Graphs & Plots

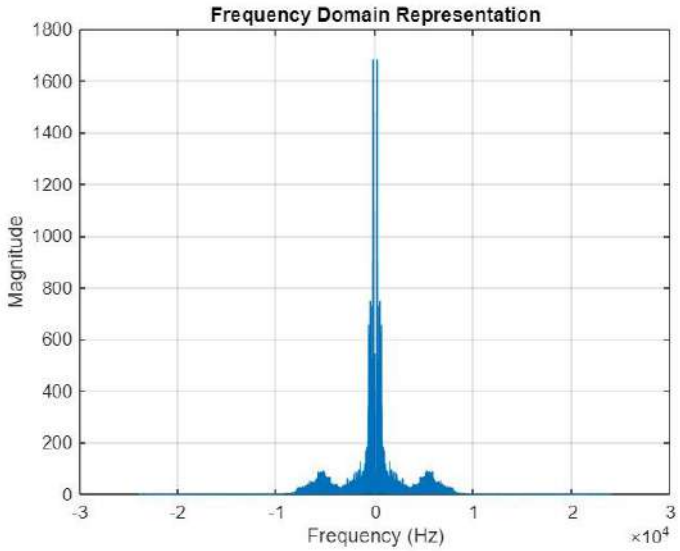
1. Original Audio Signal (Time Domain)

This figure shows the original audio recording in the time domain. It represents the sound amplitude of the group members' names over a 15-second duration. The waveform is centered around zero, indicating a balanced audio signal.



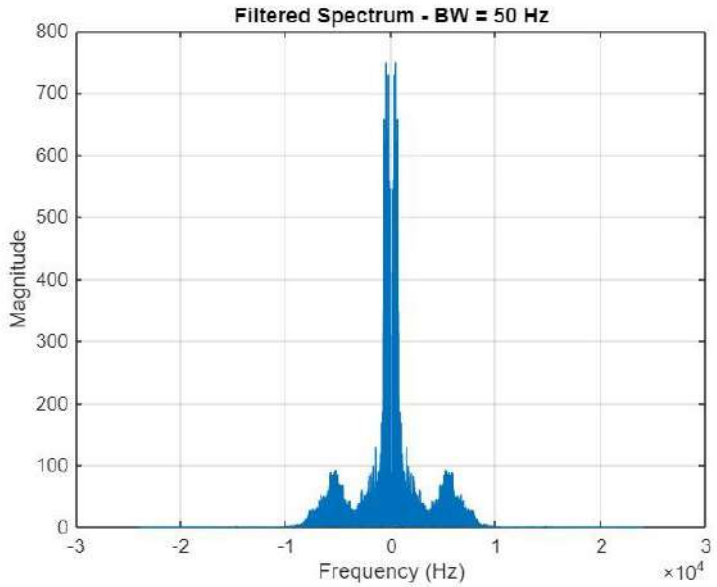
2. Original Frequency Spectrum

The frequency spectrum reveals the distribution of signal power across different frequencies. A significant peak is observed near 0 Hz, which is characteristic of human speech. The symmetry in the plot is a result of the FFT properties for real-valued signals.



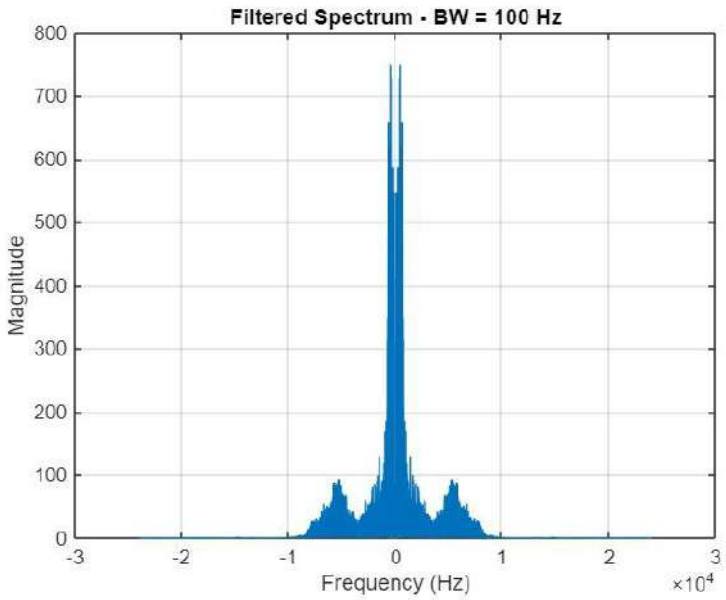
3. Filtered Spectrum (BW = 50 Hz)

In this plot, an ideal notch filter with a narrow bandwidth of 50 Hz has been applied. We can see a sharp 'cut' at the peak frequency F_{peak}. This narrow notch removes a very specific tone while preserving most of the original audio quality.



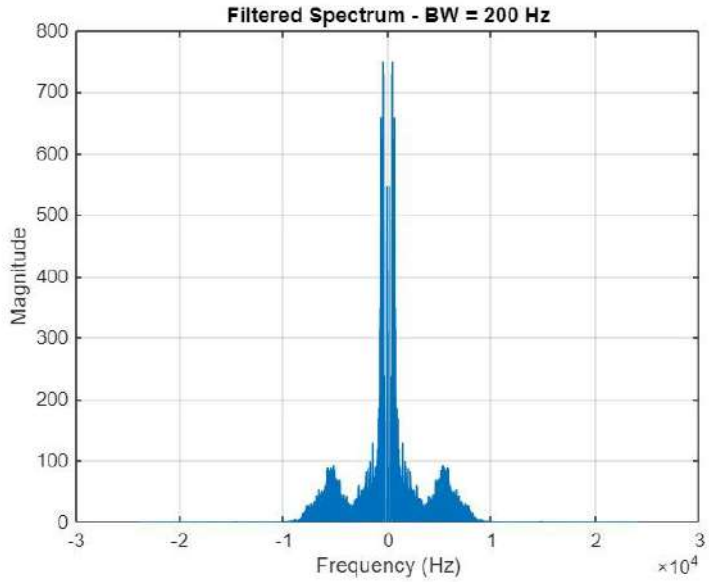
4. Filtered Spectrum (BW = 100 Hz)

By increasing the bandwidth to 100Hz, the notch filter removes a wider range of frequencies around the peak. This illustrates the trade-off between removing the dominant tone and potentially affecting the natural sound of the voices.



5. Filtered Spectrum (BW = 200 Hz)

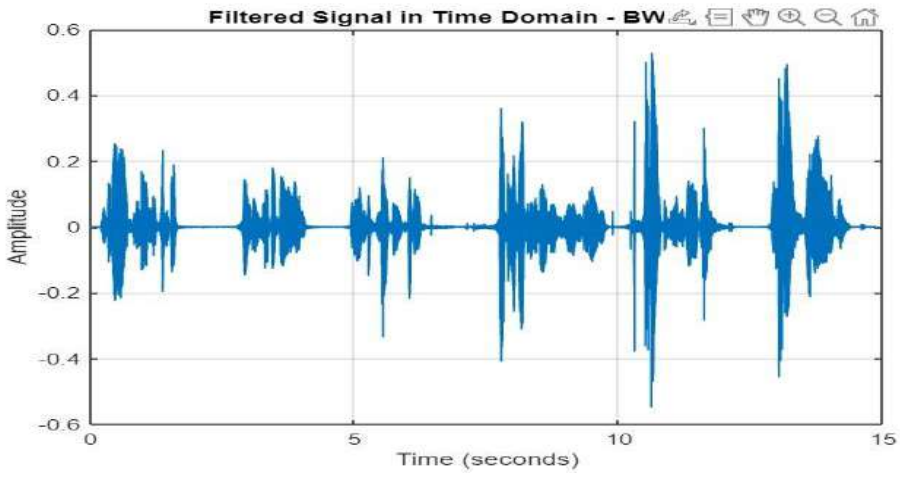
This figure shows the effect of a wide 200Hz notch. The gap in the spectrum is now clearly visible. Removing such a wide band significantly alters the frequency content, which will result in a more noticeable distortion in the reconstructed audio.



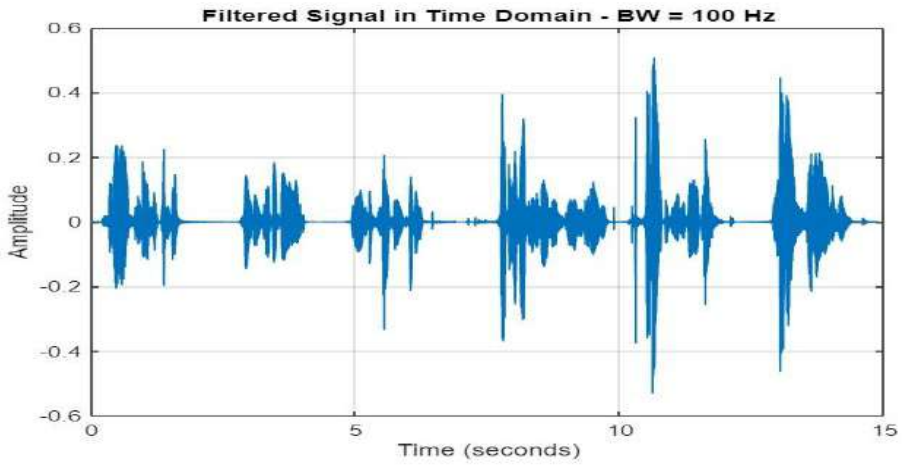
6. Filtered Signal in Time Domain (Comparison)

These plots represent the signal after being transformed back to the time domain for each bandwidth (50, 100, 200 Hz). While the general shape of the speech remains similar, the subtle changes in amplitude peaks reflect the loss of the filtered frequency components.

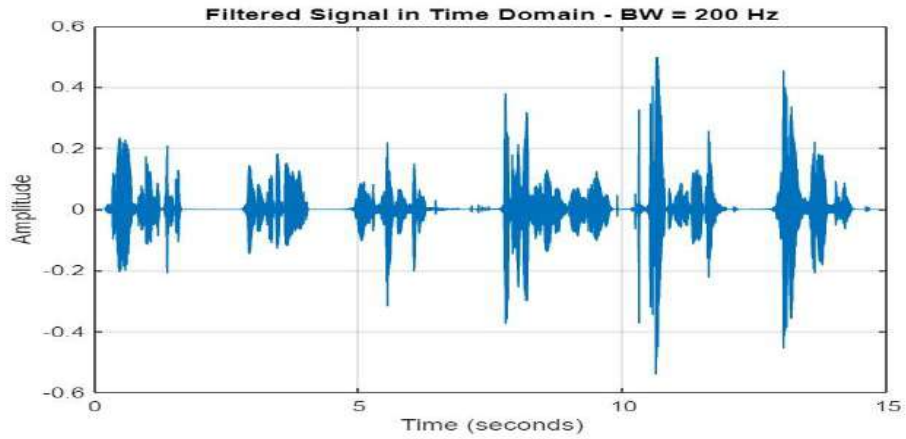
BW = 50Hz



BW = 100 Hz



BW = 200 Hz



Conclusion

In this project, the audio signal was successfully analyzed in both time and frequency domains using MATLAB. The signal energy was calculated in the two domains, and the results verified Parseval's theorem since both energies were nearly equal. The FFT was used to identify the peak frequency component, then ideal notch filters with different bandwidths were applied to remove that frequency. Finally, the filtered signals were converted back to the time domain and saved as new audio files. The project demonstrated the practical use of FFT, frequency analysis, and digital filtering in signal processing.

Project Drive Link